

Reply Networks on Bluesky: Structure, Communities and Sentiment of Political Discussions

Lorenzo Cascone

l.cascone2@studenti.unipi.it

Student ID: XXXXXXXX

Salvatore Puccio

s.puccio1@studenti.unipi.it

Student ID: XXXXXXXX

ABSTRACT

In this project we collected reply threads about US politics from Bluesky and built a network where two users are connected if one replied to the other. Every edge also has a sentiment score computed with a RoBERTa model trained on tweets. The final network has 10,855 nodes and 13,462 undirected edges. We compared it with Erdős-Rényi and Barabási-Albert graphs of the same size: the degree distribution is heavy-tailed, the average distance is short and the clustering is higher than in the random graph. Then we compared four community detection algorithms and used the sentiment of the edges to describe the communities found by Leiden: almost all of them have a negative average tone.

1

KEYWORDS

Social Network Analysis, Bluesky, Community Detection, Sentiment Analysis

1 INTRODUCTION

Bluesky is a micro-blogging platform built on the decentralised AT Protocol. Differently from other platforms, its data are public and can be collected through an open API, which makes it a good source for network studies [8].

The goal of our project is to study how users reply to each other when they discuss US politics, and whether the tone of these replies has a relation with the structure of the network. In particular, we are interested in echo chambers [4], i.e. groups of users that mostly interact among themselves. This question will be addressed in the open question of the project; in this report we describe the data collection (Section 2), the analysis of the network compared with random graphs (Section 3) and the community detection task (Section 4).

2 DATA COLLECTION

Selected Data Sources

We used the public API of Bluesky through the atproto Python client. We only downloaded public posts and replies; no follower lists or private messages were collected.

In our network a node is a Bluesky account and a directed edge (u, v) means that u replied at least once to a post written by v . For every edge we saved:

- `comments_list`: the texts of all the replies from u to v ;
- `weight`: the number of these replies;
- `sentiment_roberta`: the average sentiment of the replies (added later, see Section 2).

Since GEXF does not support lists as attributes, we stored the list of comments as a string, which is converted back to a list when we read the file.

Crawling Methodology and Assumptions

The crawl was run on 10 February 2026. The code is in `script_v3.py`, which is the cleaned-up version of the script we used, with the same parameters. The crawler works in two phases.

Phase 1. For each of the three queries *trump*, *biden* and *politics* we searched up to 100 top posts in English (`sort=top`, `lang=en`). For each of these seed posts we downloaded the thread with `get_post_thread` up to depth 3, i.e. replies, replies to replies and replies to those.

Phase 2. From the users that replied in Phase 1 we took 100 accounts and, for each of them, we searched their 5 top posts (`query from:handle`) and downloaded those threads with the same depth. In the run used for this dataset the 100 accounts were simply the first 100 elements of a Python set, so the selection was arbitrary and would change in a new run; after the crawl we changed the script to sort the handles, so that the selection is reproducible.

A recursive function goes down each thread and, for every reply, adds the text to the list of the pair (author of the reply, author of the parent post). We did not save self-replies, but we kept going down the thread after them, so that the replies below are not lost. Replies shorter than 5 characters were discarded, and handles were normalised by removing the default `.bsky.social` suffix. To respect the rate limits we waited 1.5 s between two thread requests.

Limitations of the sampling. This procedure is not a random sample of Bluesky. Starting from the *top* posts, we select the most popular threads, which usually receive many replies. As a consequence, the authors of the seed posts become hubs with hundreds of neighbours, and part of the

¹Project Repository

https://github.com/savato0/2026_puccio_cascone

heavy tail of the degree distribution and of the short distances that we find in Section 3 is probably caused by the crawling strategy. For the same reason the replies may be more polarised than an average Bluesky conversation. Moreover, the data were collected in a single day and with three political queries only.

Sentiment Enrichment

The script `graph_sentiment_roberta.py` computes the sentiment of every reply with the model `cardiffnlp/twitter-roberta-base-sentiment-latest`, a RoBERTa model trained on tweets and fine-tuned for sentiment classification [9]. For each reply the model returns the probabilities of the classes negative, neutral and positive; we use the score $s = p_{pos} - p_{neg}$, which goes from -1 to $+1$. The sentiment of an edge is the mean score of its replies. Bluesky posts have at most 300 characters, so the 512-token limit of the model is never a problem.

Data Understanding

Before the analysis we checked the quality of the 14,454 replies (notebook `dataUnderstanding.ipynb`). The median length is 71 characters; 10.5% of the replies are shorter than 20 characters and 64.3% have at least 50 characters. Almost all the pairs of users (97.2%) exchanged a single reply, and the maximum weight is 9.

The sentiment of the edges is mostly negative: the mean is -0.39 and the median -0.58 ; 54.2% of the edges have a score below -0.5 and only 8.9% above $+0.5$. We have to remember that this score measures the tone of the text and not the relation between the two users: two users who agree while criticising a politician both get a negative score.

Final Dataset

The directed network has 10,855 nodes and 14,011 edges, without self-loops. For the analysis we use the undirected, unweighted and simple version of the network: the 549 pairs of users who replied to each other become a single edge, so the undirected graph has 13,462 edges. When two directed edges are merged, their sentiment is averaged using the number of replies as weight.

3 NETWORK CHARACTERIZATION

We compared our network with two random graphs with the same number of nodes $N = 10,855$:

- an Erdős-Rényi (ER) graph [7] with $p = 2M/(N(N-1)) \approx 2.29 \cdot 10^{-4}$, so that the expected number of edges is the same as ours;
- a Barabási-Albert (BA) graph [2]. The parameter m must be an integer and $M/N \approx 1.24$, so we used $m = 1$. With $m = 1$ the BA graph is a tree with 10,854 edges,

about 19% fewer than our graph, and it cannot have triangles.

The basic statistics are reported in Table 1.

Table 1: Basic statistics of the Bluesky network and of the two random graphs.

	Bluesky	ER	BA
N	10,855	10,855	10,855
M	13,462	13,439	10,854
density	$2.29 \cdot 10^{-4}$	$2.28 \cdot 10^{-4}$	$1.84 \cdot 10^{-4}$
avg. degree	2.48	2.48	2.00
std. degree	14.71	1.56	4.26
max degree	916	11	307

Degree Distribution

The average degree is the same for Bluesky and ER, but the standard deviation and the maximum degree are very different: in ER no node has more than 11 neighbours, while in our network the most connected account has 916. BA is in between. Figure 1 shows the degree distribution in linear and log-log scale, and Figure 2 the complementary cumulative distribution (CCDF), which is easier to read for the tail.

In log-log scale the Bluesky distribution looks almost like a straight line, so we fitted a power law $P(k) \propto k^{-\alpha}$ with the `powerlaw` library [1], which uses maximum likelihood [5]. When the library chooses x_{min} automatically (the value that minimises the Kolmogorov-Smirnov distance) it selects $x_{min} = 1$, so the fit uses all the nodes, and we obtain $\alpha \approx 2.46$. We then compared the power law with other distributions using the likelihood ratio test of the library ($R > 0$ means that the power law fits better). The power law is better than the exponential ($R = 9.4$, $p \approx 4.3 \cdot 10^{-21}$) and the lognormal ($R = 13.7$, $p \approx 5.5 \cdot 10^{-43}$), while the difference with a truncated power law is not significant ($R = 0.02$, $p = 0.99$), so we cannot say which of the two is better. The ER distribution is concentrated around the mean, as expected for a Poisson distribution, while BA has the typical power-law tail of preferential attachment.

Connected Components

Our network has 65 connected components. The giant component (GCC) contains 10,595 nodes (97.6%), the second one only 17, and the other 63 components have 243 nodes in total. These are most likely threads whose users do not appear anywhere else in the sample. The ER graph with the same density is much more fragmented: 1,026 components, a GCC with 89.0% of the nodes and 892 isolated nodes. The BA graph is connected by construction. The following analyses

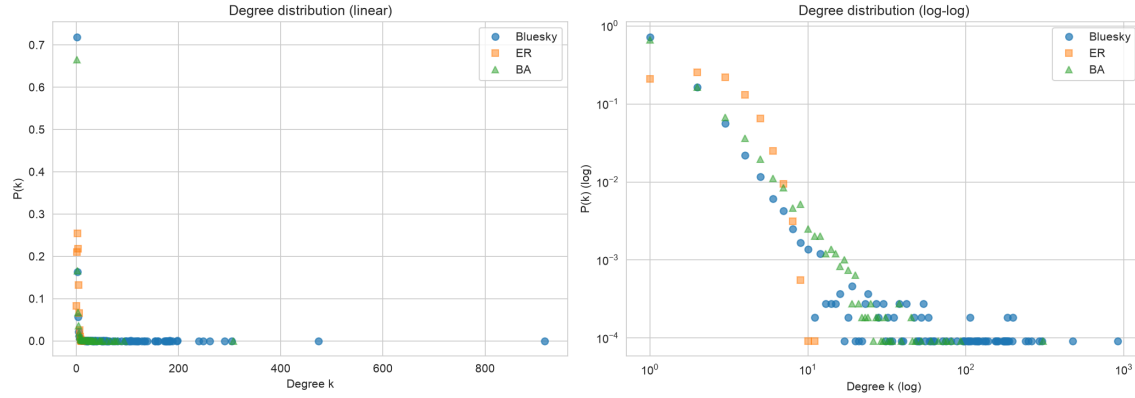


Figure 1: Degree distribution of Bluesky, ER and BA in linear (left) and log-log scale (right).

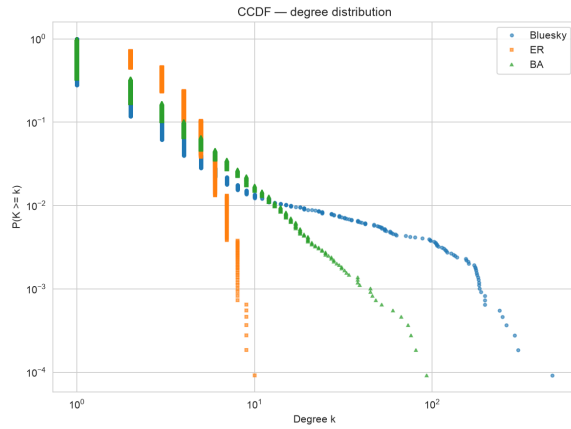


Figure 2: CCDF of the degree in log-log scale.

of paths and centralities are computed on the GCC of each graph.

Path Analysis

Since the graphs are not too big, we computed the exact average shortest path length and diameter with a BFS from every node of the GCC. The results are:

- Bluesky: average distance 5.03, diameter 18;
- ER: average distance 10.00, diameter 21;
- BA: average distance 8.86, diameter 23.

Our network has the same density as ER, but its distances are about half. The reason is the presence of hubs: many users are connected to the same few accounts, so most paths can go through them. Together with the clustering discussed below, this is the typical behaviour of a small-world network. We have to remember, however, that these hubs are partly created by the way we collected the data.

Clustering and Density

The density of our network is very low ($2.29 \cdot 10^{-4}$) and the same as ER, as expected. The clustering coefficients are reported in Table 2.

Table 2: Average clustering coefficient and transitivity.

	Bluesky	ER	BA
avg. clustering	0.0325	0.0004	0
transitivity	0.0011	0.0007	0

The average clustering is almost 100 times higher than in ER, so in our network there are more triangles than in a random graph: sometimes, if two users reply to the same account, they also reply to each other. The transitivity, instead, is only slightly higher than in ER. The two measures are different because the average clustering gives the same weight to every node, while the transitivity counts connected triples. Only 740 nodes are part of at least one triangle, and 76.6% of them have degree 5 or less, so their local clustering is high. On the other hand, 95.8% of the connected triples are centred on the 50 nodes with the highest degree, whose neighbours are almost never connected to each other (the local clustering of ronfilipkowski is 0.0002). So the triadic closure in our network is present, but weak and limited to a small part of the users.

Centrality Analysis

On the GCC we computed five centrality measures.

Degree. Among the accounts with more neighbours there are journalists, news outlets and political commentators. The top ten are ronfilipkowski (916), cortneymckee (474), atrupar.com (304), forbes.com (290), thetnholler (261), reichlinmelnick (248), jonbandit

(239), womenriseup2026 (198), carlquintanilla (198) and patriottakes (197).

Closeness. The ranking is similar to the degree one, but not the same: for example rbreich and nothoodlum are in the top six, even if they are not among the nodes with the highest degree.

Betweenness. To save time we estimated it from $k = 500$ random source nodes. It is very concentrated: ronfilipkowski has 0.27, more than twice the second node (cortneymckee, 0.12), so a large part of the shortest paths goes through this account.

Eigenvector centrality and PageRank. We computed eigenvector centrality with the power method and PageRank with damping factor 0.85. Eigenvector centrality is almost all on ronfilipkowski (0.70), while all the other nodes are below 0.09: this happens because the score of a node depends on the score of its neighbours, and the neighbours of the biggest hub get most of the value. PageRank, thanks to the random jumps, is more spread over the network.

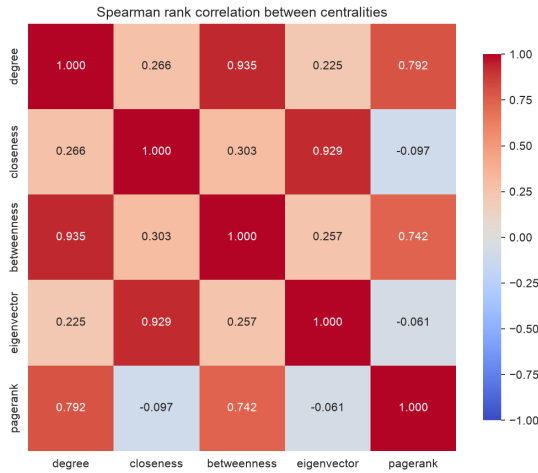


Figure 3: Spearman correlation between the five centrality measures on the GCC.

Correlation. Figure 3 shows the Spearman correlation between the measures. Degree, betweenness and PageRank are strongly correlated (between 0.74 and 0.94), so the accounts with more replies are also the ones on most shortest paths. Closeness and eigenvector centrality are strongly correlated with each other (0.93) but only weakly with degree and betweenness (0.23-0.30), and PageRank is not correlated with them (-0.06 with eigenvector, -0.10 with closeness). The reason is that closeness and eigenvector centrality are high for all the nodes near ronfilipkowski, also the ones with degree 1, while degree and PageRank depend on the number of neighbours of the node itself. In fact, 7,597 nodes of the GCC have degree 1, and 581 of them are connected

only to ronfilipkowski: compared with the other degree-1 nodes, these have a higher median closeness (0.24 vs 0.20) and a much higher median eigenvector centrality (0.023 vs 0.0002), but the same PageRank.

Comparison with ER and BA

Overall, our network is sparse, has a giant component with almost all the nodes, a heavy-tailed degree distribution and short distances. ER reproduces the density but not the hubs, the distances and the clustering. BA has hubs and a heavy tail, but with $m = 1$ it has fewer edges, longer distances than our network and no triangles. So neither of the two models describes our network well: the hubs are closer to BA, the clustering is higher than both.

4 TASK 1: COMMUNITY DETECTION

Our group has two members, so we had to choose one task of Part 3. We chose community detection because the GCC contains almost all the nodes and because the sentiment of the edges can help us to describe the communities. All the algorithms run on the unweighted GCC ($N = 10,595$, $M = 13,263$); the sentiment is only used after, to interpret the results.

Algorithms

We used four algorithms implemented in cdlib [11]:

- **Louvain** [3]: every node is moved to the neighbouring community that gives the largest increase of modularity; then the communities are merged into single nodes and the procedure is repeated.
- **Leiden** [13]: an improvement of Louvain that guarantees that every community is connected.
- **Infomap** [12]: it looks for the partition that gives the shortest description of a random walk on the network. We used the `--two-level` option to get a flat partition like the other methods.
- **Label Propagation** [10]: every node takes the most frequent label among its neighbours until the labels do not change. We used the semi-synchronous version [6], which always gives the same result.

In a first run with other library versions Infomap and Label Propagation gave different results, because the default options had changed. For this reason we set these options explicitly, fixed the seed of Leiden and wrote the versions of the libraries in the requirements of the repository.

Internal Evaluation

For each partition we computed the number of communities, the size of the largest and of the median community, the Newman-Girvan modularity Q , the average conductance (for each community, the fraction of its edges that go outside;

lower is better) and the internal edge density (internal edges divided by all the possible pairs of nodes in the community). The results are in Table 3.

Table 3: Internal evaluation of the four partitions on the GCC.

Algorithm	#com.	largest	median	Q	cond.	density
Louvain	60	747	165	0.819	0.15	0.02
Leiden	60	740	162	0.820	0.15	0.02
Infomap	498	592	6	0.772	0.29	0.42
LabelProp	931	863	2	0.734	0.37	0.74

Louvain and Leiden give very similar results: 60 communities of a few hundred nodes, modularity 0.82 and low conductance. Such a high modularity means that the network has a clear community structure. Infomap and Label Propagation instead find hundreds of small communities (median size 6 and 2), with higher conductance. Their internal density is higher only because small communities have few possible pairs of nodes. Their modularity is also lower, but this comparison is not completely fair, because these two methods do not optimise modularity.

Comparison between Algorithms

To see how similar the partitions are, we computed the Normalized Mutual Information (NMI) and the Adjusted Rand Index (ARI) for every pair of algorithms (Figure 4). Both are 1 when two partitions are identical; ARI is corrected for chance and is close to 0 for random partitions. Louvain and Leiden are very similar (NMI 0.90, ARI 0.85), and so are Infomap and Label Propagation (NMI 0.88, ARI 0.70). Between the two groups the NMI is still quite high (0.76-0.85), while the ARI is lower (0.54-0.64). Since NMI tends to be high when one of the partitions has many small communities, we give more weight to the ARI. Our interpretation is that Infomap and Label Propagation split the communities of Louvain and Leiden into smaller groups, but we did not verify this directly.

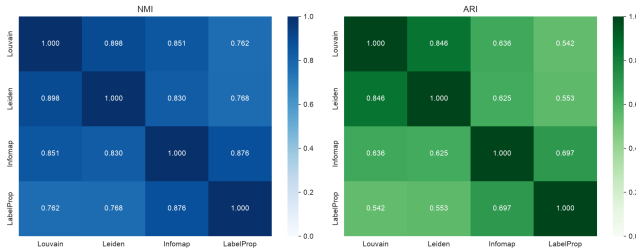


Figure 4: NMI (left) and ARI (right) between the four partitions.

Communities and Sentiment

For the interpretation we used the Leiden partition, which has the highest modularity (0.8197, practically the same as Louvain with 0.8186). For each community we looked at the nodes with the highest degree and we computed the mean sentiment of the edges inside the community. Table 4 shows some of the largest communities and the only one with a positive mean.

Table 4: Some Leiden communities: size, fraction of internal edges, mean sentiment of the internal edges and the two nodes with the highest degree.

size	internal	sentiment	main nodes
740	0.73	-0.42	ronfilipkowski, patriotclyde
380	0.66	-0.32	cortneymckee, h0pe4dfuture
279	0.73	-0.42	carlkuintanilla, billkristolbulwark
274	0.69	-0.59	powerbolt, ky-blue17
257	0.75	-0.51	reichlinmelnick, cullenskink
249	0.69	-0.55	atrupar.com, tnsteve.com
243	0.71	-0.56	forbes.com, mayflowerdescndnt
197	0.83	+0.08	phillewis, snelsonus

Many of the largest communities contain one of the hubs found in Section 3, which is expected since we collected whole threads starting from popular posts. In the 30 largest communities the fraction of edges that stay inside the community goes from 0.65 to 0.89, so users mostly reply to users of the same community.

The sentiment is negative almost everywhere (Figure 5): 59 of the 60 communities have a negative mean, and the mean over the communities is -0.37 (median -0.38). The only exception is the community around phillewis, with a mean of $+0.08$. Moreover, larger communities tend to be more negative (Spearman $\rho = -0.41$, $p < 0.01$; Pearson $r = -0.27$, $p = 0.04$).

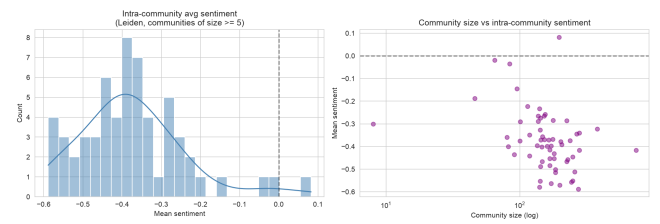


Figure 5: Left: distribution of the mean sentiment of the Leiden communities. Right: community size and mean sentiment.

These results must be read carefully. A negative sentiment inside a community does not mean that its members are hostile to each other: as said in Section 2, the model measures the tone of the text, and users who agree with each other

can write negative replies about the same political topic. For this reason the sentiment inside the communities alone does not tell us if they are echo chambers. To study this we need to compare the edges inside the communities with the edges between different communities, which is what we plan to do in the open question.

5 DISCUSSION

We built a reply network of 10,855 Bluesky users from political threads. Compared with random graphs of the same size, it has a heavy-tailed degree distribution ($\alpha \approx 2.46$), average distances that are about half of ER, a clustering higher than ER and a clear community structure ($Q \approx 0.82$). Many of the largest communities contain one of the most popular accounts, and almost all the communities have a negative tone.

The main limitation of our work is the sampling. We started from the top posts of three political queries, collected in a single day, and expanded the network from 100 users. This probably makes the hubs bigger and the distances shorter than in the whole platform, and it can also make the sentiment more negative. Other limitations are the BA model with $m = 1$, which has fewer edges than our network, the approximated betweenness and the use of a sentiment model trained on Twitter data. Finally, the results of community detection depend on the options and on the versions of the libraries, so we fixed both in our code.

REFERENCES

- [1] Jeff Alstott, Ed Bullmore, and Dietmar Plenz. 2014. powerlaw: A Python Package for Analysis of Heavy-Tailed Distributions. *PLoS ONE* 9, 1 (2014), e85777.
- [2] Albert-László Barabási and Réka Albert. 1999. Emergence of scaling in random networks. *Science* 286, 5439 (1999), 509–512.
- [3] Vincent D. Blondel, Jean-Loup Guillaume, Renaud Lambiotte, and Etienne Lefebvre. 2008. Fast unfolding of communities in large networks. *Journal of Statistical Mechanics: Theory and Experiment* 2008, 10 (2008), P10008.
- [4] Matteo Cinelli, Gianmarco De Francisci Morales, Alessandro Galeazzi, Walter Quattrociocchi, and Michele Starnini. 2021. The echo chamber effect on social media. *Proceedings of the National Academy of Sciences* 118, 9 (2021), e2023301118.
- [5] Aaron Clauset, Cosma Rohilla Shalizi, and M. E. J. Newman. 2009. Power-law distributions in empirical data. *SIAM Rev.* 51, 4 (2009), 661–703.
- [6] Gennaro Cordasco and Luisa Gargano. 2010. Community detection via semi-synchronous label propagation algorithms. In *2010 IEEE International Workshop on Business Applications of Social Network Analysis (BASNA)*.
- [7] Paul Erdős and Alfréd Rényi. 1959. On random graphs I. *Publicationes Mathematicae Debrecen* 6 (1959), 290–297.
- [8] Andrea Failla and Giulio Rossetti. 2024. “I’m in the Bluesky Tonight”: Insights from a year worth of social data. *PLOS ONE* 19, 11 (2024), e0310330. doi:10.1371/journal.pone.0310330
- [9] Daniel Loureiro, Francesco Barbieri, Leonardo Neves, Luis Espinosa Anke, and Jose Camacho-Collados. 2022. TimeLMs: Diachronic Language Models from Twitter. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*. 251–260.
- [10] Usha Nandini Raghavan, Réka Albert, and Soundar Kumara. 2007. Near linear time algorithm to detect community structures in large-scale networks. *Physical Review E* 76 (2007), 036106.
- [11] Giulio Rossetti, Letizia Milli, and Rémy Cazabet. 2019. CDLIB: a python library to extract, compare and evaluate communities from complex networks. *Applied Network Science* 4, 1 (2019), 52.
- [12] Martin Rosvall and Carl T. Bergstrom. 2008. Maps of random walks on complex networks reveal community structure. *Proceedings of the National Academy of Sciences* 105, 4 (2008), 1118–1123.
- [13] V. A. Traag, L. Waltman, and N. J. van Eck. 2019. From Louvain to Leiden: guaranteeing well-connected communities. *Scientific Reports* 9 (2019), 5233.